# Zadanie 1. Heurystyki konstrukcyjne

Inteligentne Metody Optymalizacji

Maciej Szymaniak, 155830
Piotr Szymaczek, 155889

23 marca 2026

## 1 Opis zadania

Rozważamy zmodyfikowany problem komiwojażera. Dany jest zbiór $n$ wierzchołków z symetryczną macierzą odległości $d_{ij}$ oraz zyskiem $p_i$ za odwiedzenie wierzchołka $i$. Należy wybrać podzbiór wierzchołków i ułożyć zamkniętą trasę przechodzącą przez wybrane wierzchołki, maksymalizując funkcję celu:

$$f(\mathcal{C}) = \sum_{i \in \mathcal{C}} p_i \; - \sum_{(i,j) \in \mathcal{C}} d_{ij}$$

Każdy algorytm składa się z dwóch faz:

- **Faza I** – budowa cyklu Hamiltona.
- **Faza II** – iteracyjne usuwanie wierzchołków, dopóki poprawia to wartość funkcji celu.

Kod źródłowy dostępny pod adresem:
https://github.com/masz00/Intelligent-Optimization-Methods
Rozwiązania zweryfikowane za pomocą SolutionCheckera: *TAK*.

## 2 Pseudokody algorytmów

### 2.1 Algorytm losowy

**Algorithm 1:** Rozwiązanie losowe

**Wejście:** wierzchołek startowy $s$, zbiór wierzchołków *punkty*
**Wyjście:** losowo wygenerowana trasa $\mathcal{C}$

$max_wierzcholki \leftarrow$ rozmiar($punkty$),
$wielkosc_trasy \leftarrow$ losowa liczba z $[1, max_wierzcholki]$;
$\mathcal{C} \leftarrow [0, 1, \ldots, max_wierzcholki - 1]$;
Przetasuj losowo elementy $\mathcal{C}$ od indeksu 1 do końca;
Skróć $\mathcal{C}$ z powrotem do wielkości równej $wielkosc_trasy$;
**zwróć** $\mathcal{C}$;

## 2.2 Nearest Neighbor (NN)

**Algorithm 2:** Nearest Neighbor – Faza I

**Wejście:** wierzchołek startowy $s$, flaga $z_zyskiem$, odległości $dystans$, zyski $wagi$
**Wyjście:** cykl Hamiltona $\mathcal{C}$ na wszystkich wierzchołkach

```
C ← [s],    odwiedzone ← {s};
dopóki |C| < n wykonaj
    ostatni ← ostatni wierzchołek w C,    najlepszy_kandydat ← −1,
     minimalny_koszt ← ∞ ;
    dla każdego v ∉ odwiedzone wykonaj
        koszt ← dystans[ostatni][v];
        jeżeli z_zyskiem to
            koszt ← koszt − wagi[v];
        koniec
        jeżeli koszt < minimalny_koszt to
            minimalny_koszt ← koszt,    najlepszy_kandydat ← v;
        koniec
    koniec
    Dodaj najlepszy_kandydat na końiec C i do odwiedzone;
koniec
zwróć C;
```

Wariant **NNa**: $z_zyskiem = false$.  Wariant **NNb**: $z_zyskiem = true$.

## 2.3 Greedy Cycle (GC)

**Algorithm 3:** Greedy Cycle – Faza I

**Wejście:** wierzchołek startowy $s$, flaga $z_zyskiem$, odległości $dystans$, zyski $wagi$
**Wyjście:** cykl Hamiltona $\mathcal{C}$ na wszystkich wierzchołkach

```
C ← [s],   odwiedzone ← {s},   drugi_wezel ← −1,
  najmniejszy_koszt_startowy ← ∞;
// Dodaj drugi wierzchołek
dla każdego v ≠ s wykonaj
    koszt ← dystans[s][v];
    jeżeli z_zyskiem to
        koszt ← koszt − wagi[v];
    koniec
    jeżeli koszt < najmniejszy_koszt_startowy to
        najmniejszy_koszt_startowy ← koszt,   drugi_wezel ← v;
    koniec
koniec
Dodaj drugi_wezel do C i do odwiedzone;
dopóki |C| < n wykonaj
    pozycja_wstawienia ← 0,   wybrany_kandydat ← −1,
      globalny_najmniejszy_koszt ← ∞;
    dla każdego indeks i oraz wierzchołki a, b współdzielące krawędź w C wykonaj
        dla każdego v ∉ odwiedzone wykonaj
            koszt_wstawienia ← dystans[a][v] + dystans[v][b] − dystans[a][b];
            jeżeli z_zyskiem to
                koszt_wstawienia ← koszt_wstawienia − wagi[v];
            koniec
            jeżeli koszt_wstawienia < globalny_najmniejszy_koszt to
                globalny_najmniejszy_koszt ← koszt_wstawienia,
                  wybrany_kandydat ← v,   pozycja_wstawienia ← i;
            koniec
        koniec
    koniec
    Wstaw wybrany_kandydat do C po wierzchołku z pozycji pozycja_wstawienia
      i dodaj do odwiedzone;
koniec
zwróć C;
```

Wariant **GCa**: $z_zyskiem = false$. Wariant **GCb**: $z_zyskiem = true$.

## 2.4 Heurystyka 2-żalu (Regret i Ważony Regret)

Zarówno standardowy algorytm 2-Regret, jak i Ważony 2-Regret (WRegret) zrealizowano za pomocą jednego, uogólnionego algorytmu operującego na wagach $w_1$ oraz $w_2$.

**Algorithm 4:** Rozbudowa z żalem (Regret i Ważony Regret) – Faza I

**Wejście:** wierzchołek startowy $s$, flaga *z_zyskiem*, odległości *dystans*, zyski *wagi*, wagi $w_1$, $w_2$

**Wyjście:** cykl Hamiltona $\mathcal{C}$ na wszystkich wierzchołkach

Zainicjalizuj $\mathcal{C}$ tak samo jak w Greedy Cycle (znajdź drugi wierzchołek i dodaj go do cyklu);

**dopóki** $|\mathcal{C}| < n$ **wykonaj**

  *najwyzsza_ocena* $\leftarrow -\infty$, *wybrany_kandydat* $\leftarrow -1$, *docelowa_pozycja* $\leftarrow -1$;

  **dla każdego** $v \notin$ *odwiedzone* **wykonaj**

    *najmniejszy_koszt* $\leftarrow \infty$, *drugi_najmniejszy* $\leftarrow \infty$, *najlepsza_pozycja* $\leftarrow 0$;

    **dla każdego** *indeks i oraz wierzchołki* $a, b$ *współdzielące krawędź w* $\mathcal{C}$ **wykonaj**

      *koszt* $\leftarrow$ *dystans*$[a][v]$ + *dystans*$[v][b]$ − *dystans*$[a][b]$;

      **jeżeli** *z_zyskiem* **to**

        *koszt* $\leftarrow$ *koszt* − *wagi*$[v]$;

      **koniec**

      **jeżeli** *koszt* < *najmniejszy_koszt* **to**

        *drugi_najmniejszy* $\leftarrow$ *najmniejszy_koszt*, *najmniejszy_koszt* $\leftarrow$ *koszt*, *najlepsza_pozycja* $\leftarrow i$;

      **koniec**

      **w p.p. jeżeli** *koszt* < *drugi_najmniejszy* **to**

        *drugi_najmniejszy* $\leftarrow$ *koszt*;

      **koniec**

    **koniec**

    *oceniony_wynik* $\leftarrow$ $w_1 \cdot$ (*drugi_najmniejszy* − *najmniejszy_koszt*) − $w_2 \cdot$ *najmniejszy_koszt*;

    **jeżeli** *oceniony_wynik* > *najwyzsza_ocena* **to**

      *najwyzsza_ocena* $\leftarrow$ *oceniony_wynik*, *wybrany_kandydat* $\leftarrow v$, *docelowa_pozycja* $\leftarrow$ *najlepsza_pozycja*;

    **koniec**

  **koniec**

  Wstaw *wybrany_kandydat* do $\mathcal{C}$ po wierzchołku z pozycji *docelowa_pozycja* i dodaj do *odwiedzone*;

**koniec**

**zwróć** $\mathcal{C}$;

Wariant **2-Regret**: $w_1 = 1,\ w_2 = 0$. Wariant **Ważony 2-Regret**: $w_1 = 1,\ w_2 = 1$. Litera 'a' na końcu skrótu nazwy oznacza brak uwzględniania zysków, a litera 'b' - uwzględnienie ich.

### 2.5 Faza II – usuwanie wierzchołków

**Algorithm 5:** Faza II – iteracyjne usuwanie wierzchołków poprawiające koszt

```
Wejście: cykl C, odległości dystans, zyski wagi
Wyjście: zoptymalizowany cykl C

znaleziono_poprawe ← PRAWDA;

dopóki znaleziono_poprawe oraz |C| > 1 wykonaj
    znaleziono_poprawe ← FAŁSZ,   indeks_do_usuniecia ← −1,
     najwiekszy_spadek_kosztu ← 0;

    dla każdego indeks i w cyklu C wykonaj
        u ← wierzchołek z pozycji (i − 1) mod |C|;
        v ← C[i];
        w ← wierzchołek z pozycji (i + 1) mod |C|;

        zmiana_kosztu ← dystans[u][w] − dystans[u][v] − dystans[v][w] + wagi[v];

        jeżeli |C| = 2 to
            zmiana_kosztu ← 0 − 2 · dystans[u][v] + wagi[v];
        koniec

        jeżeli zmiana_kosztu < najwiekszy_spadek_kosztu to
            najwiekszy_spadek_kosztu ← zmiana_kosztu,   indeks_do_usuniecia ← i;
        koniec
    koniec
    jeżeli najwiekszy_spadek_kosztu < 0 to
        Usuń wierzchołek z pozycji indeks_do_usuniecia w C;
        znaleziono_poprawe ← PRAWDA;
    koniec
koniec
zwróć C;
```

Ujemna zmiana kosztu (czyli spadek zsumowanego dystansu będącego obciążeniem, a docelowo wzrost końcowej funkcji celu) oznacza, że usunięcie wierzchołka (zaoszczędzenie dwóch krawędzi, dodanie jednej, stracenie zysku z wierzchołka poprzez wymuszone dodanie jego wagi do spadku w formie kary) jest opłacalne dla ogólnego wyniku.

## 3 Wyniki eksperymentu obliczeniowego

Każdy algorytm uruchomiono 200 razy (każdy wierzchołek startowy użyty dokładnie raz). Przedstawione wyniki to uśredniona wartość funkcji celu ze wszystkich przebiegów oraz w nawiasach *(minimum – maksimum)*.

### 3.1 Faza I

Tabela 1: Funkcja celu po Fazie I

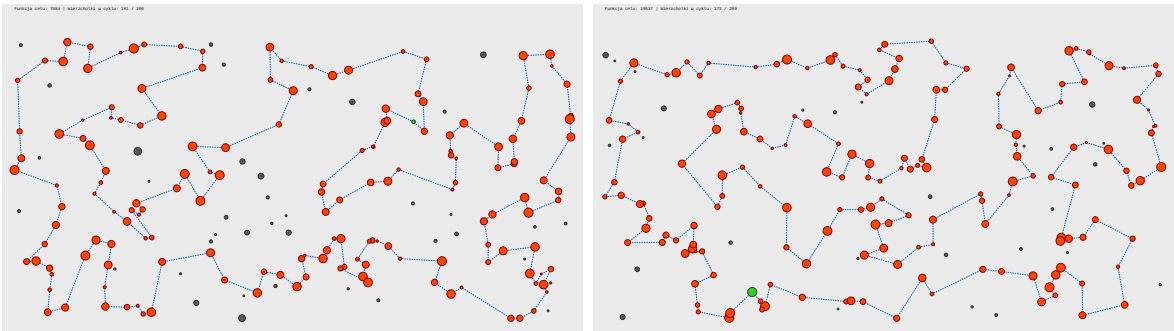
| Metoda | TSPA | TSPB |
|---|---|---|
| Random | -152884 (-329964 – -4018) | -141133 (-293035 – 463) |
| NNa | -3758 ( -8436 – -944) | 9274 ( 4704 – 11839) |
| NNb | -12193 (-14403 – -10537) | -3015 ( -7072 – 1063) |
| GCa | -1542 ( -2470 – -912) | 9707 ( 8658 – 11224) |
| GCb | -949 ( -1800 – -141) | 11560 ( 10657 – 12164) |
| RegretGCa | 2816 ( 1185 – 3666) | 15573 ( 14276 – 17071) |
| RegretGCb | 2796 ( 1927 – 3666) | 15580 ( 14445 – 16670) |
| WRegretGCa | 139 ( -544 – 1033) | 11927 ( 10413 – 13076) |
| WRegretGCb | 121 ( -528 – 853) | 12444 ( 11472 – 13451) |

### 3.2 Faza II

Tabela 2: Wartość funkcji celu po Fazie II

| Metoda | TSPA | TSPB |
|---|---|---|
| Random | -2003 (-10799 – 622) | -3107 (-13911 – 611) |
| NNa | 2555 ( -2121 – 5971) | 13383 ( 10375 – 16455) |
| NNb | -879 ( -3899 – 2712) | 5719 ( 2251 – 9016) |
| GCa | 3504 ( 1894 – 4368) | 13655 ( 12527 – 15375) |
| GCb | 3650 ( 2936 – 4506) | 15101 ( 14181 – 16103) |
| RegretGCa | 6147 ( 4928 – 6934) | 18092 ( 16877 – 19517) |
| RegretGCb | 6099 ( 4743 – 7084) | 18091 ( 16877 – 19475) |
| WRegretGCa | 3936 ( 3305 – 4768) | 15395 ( 14384 – 16506) |
| WRegretGCb | 4328 ( 3366 – 5253) | 16108 ( 15364 – 16976) |

## 4 Wizualizacje najlepszych rozwiązań



Rysunek 1: Najlepsze rozwiązania wg wyników eksperymentu: RegretGCb dla instancji TSPA (po lewej) oraz RegretGCa dla instancji TSPB (po prawej).

## 5 Wnioski

Przeprowadzony eksperyment wykazuje zdecydowaną dominację heurystyki 2-Regret, która osiągnęła najlepsze wyniki końcowe na obu testowanych instancjach. Kluczową rolę odgrywa tu Faza

II, która drastycznie poprawia funkcję celu każdego algorytmu, w tym silnie optymalizując bardzo słabe rozwiązanie losowe. Włączenie zysku do funkcji oceny wyraźnie poprawia jakość podstawowego algorytmu Greedy Cycle, jednak w przypadku najlepszej metody (RegretGC) różnice te niemal całkowicie zacierają się po Fazie II. Dodatkowo eksperyment dowiódł, że standardowy 2-Regret sprawdza się zauważalnie lepiej niż Ważony 2-Regret, a metoda zachłannego cyklu (GC) systematycznie deklasuje algorytm najbliższego sąsiada (NN) pod kątem ostatecznych wartości funkcji celu.